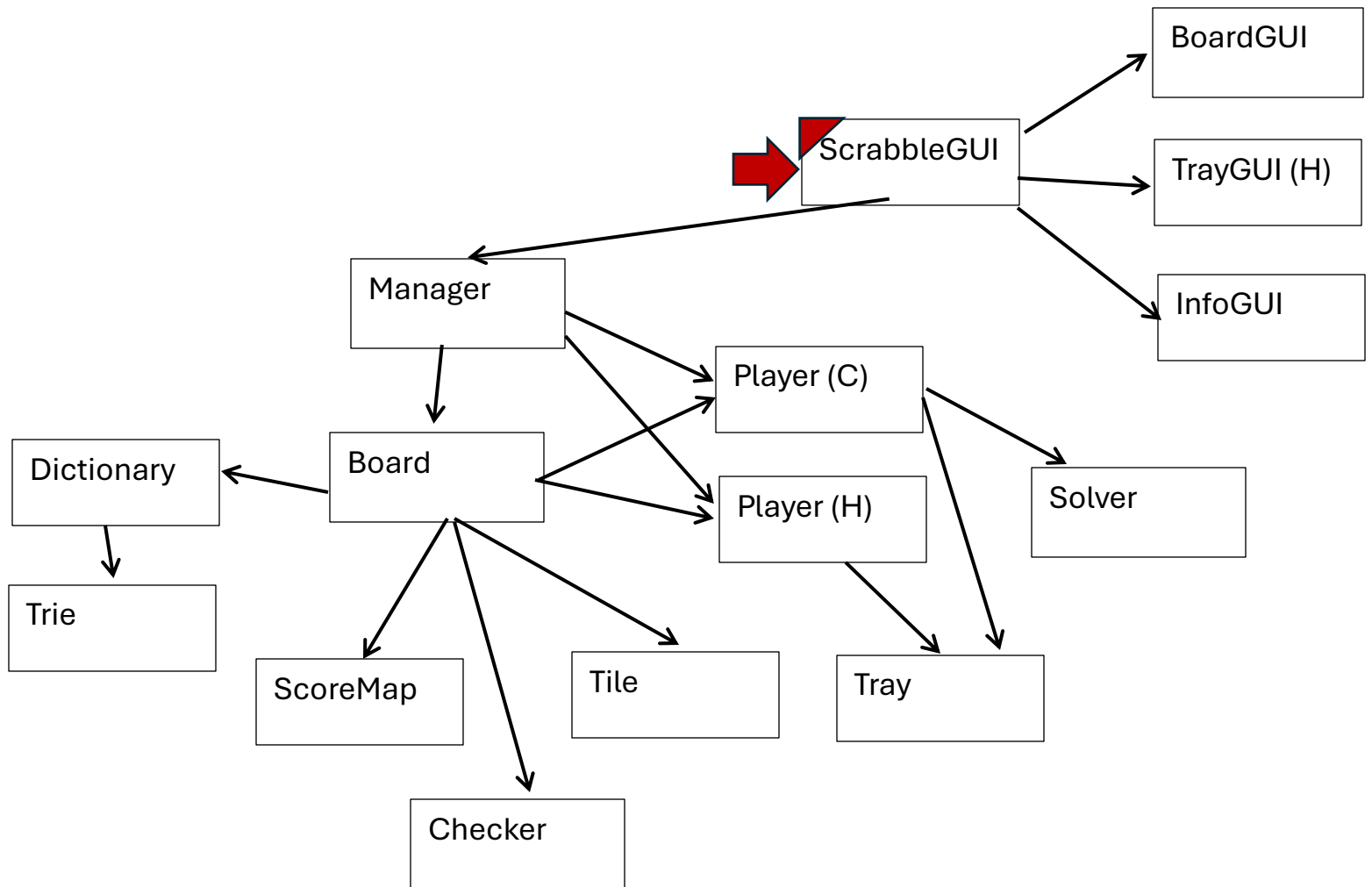




ScrabbleGUI: The Main Game Loop and Input would come from this class. Would have a Game Manager Object to get the fields needed to display the GUI elements and the manager would handle all the Game Logic

BoardGUI, TrayGUI, InfoGUI: These elements would belong to the ScrabbleGUI Class so they can get information from the Manager to create the GUI elements to display the state of the game.

- The InfoGUI would take in the Direction of the move, with a button so the manager can make the move and check the move in the correct direction, would also have a 'PassTurn' button

- The BoardGUI takes in a Coordinate that is the first coord of the word, that is passed to the manager to make a human move.

- The TrayGUI would then take the next Clicks to form the word the Human wants to play

- The InfoGUI would then end the Human turn by having a 'MakeMove' Button to then check the move. If the move is invalid the human can always use the skip button

Manager: This class would be responsible for keeping track of all the Game Elements and having Fields the ScrabbleGUI could access to represent the state of the Game. The Board, Human Tray and Scores are the things that would show on the display but would also contain the Computer Player [Player(C)] information to use in the Solver.

Board: The Scrabble board is Stored as a 2D-Array of custom Tile objects. The Board contains the objects that are the same throughout the game, so the Dictionary, ScoringMap, and the Players, then those objects can access the each other through the board methods that are implemented

Dictionary and Trie: The Dictionary is stored as a Prefix Trie [Trie] data structure, implemented using nodes that contain child nodes and a Boolean to represent the end of a valid word. The Trie is the data structure, and the Dictionary is stored as a Trie

ScoreMap: This class is used to create the maps for Scoring and Tile Frequency, which is passed in through a .txt file into the board. There is only one ScoreMap object per board, so the scoring is consistent throughout the game.

Checker: is included in the diagram but is implemented as a series of method calls for getting the score. The Checker.java code uses the same sort of structure to validate a score and the same structure is used in the solver to find the highest scoring plays.

Tile: This class is used to store all the necessary information about the tiles in the game. This includes the Tiles on the board that are empty (and whether they have multipliers, and the type of multiplier if so), played tiles and corresponding scores, and other methods like 'isEmpty()' to use in the Board class for easy access to tile information. Tiles are also used in the Trays to calculate scores at the end of the games and account for deductions if needed.

Player (H): Human Player class used for handling the Human Play. Input would come from the ScrabbleGUI into the Manager, into the Player(H), then the board can access this information to update and score the play

Player(C): Computer player class used for making a computer play. Would contain a Solver and a Tray that and would be used to find the best play and pass that to the Manager, then into the board to be updated and scored.

Solver: The Solver was my best attempt to implement the Algorithm describes in 'The World's Fastest Scrabble Program (1988)' *See Doc Folder. The Algorithm is uses Anchor Points, a Dictionary (as a Trie), and a Backtracking algorithm to find the best scoring play. The parts that are in the algorithm that I struggled to implement are the "Reduction to 1-

Dimension” and using the “Anchors” in the correct manner. Although the Algorithm gives a high scoring play, the play is often time not legal and ignore the rule that the the word must form a correct word in the adjacent direction.